

Reviewer Pack — Minimal Geometric Entropy of Projective Varieties and Commun...

Entry 5a102f9f0fa2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 01:53:08 UTC

Minimal Geometric Entropy of Projective Varieties and Communication Complexity Rank

Entry ID: 5a102f9f0fa2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 01:53:08 UTC

1. Conjecture

Field A (mathematical branch): Projective Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any given d -dimensional projective variety V with genus g , the minimal geometric entropy ($\text{mge}(V)$) is lower-bounded by the communication complexity rank ($\text{ccr}(V)$), such that $\text{mge}(V) = \Omega(\text{ccr}(V))$.

Rationale (proposer's reasoning):

Projective geometry has been used to model certain aspects of discrete structures. The geometric entropy captures the complexity of a variety's structure, while communication complexity measures the information needed for two parties to agree on some property. This conjecture posits that high-dimensional structures with complex geometries have higher communication complexities.

Taxonomy category: ProjectiveGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0a0aadcfd2c7aea9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that minimal geometric entropy ($\text{mge}(V)$) of a d -dimensional projective variety V with genus g is lower-bounded by the communication complexity rank ($\text{ccr}(V)$), support will be indicated if for all $n \leq 40$ variables, $\text{mge}(V) / \text{ccr}(V) \geq 0.8$ AND the mean of $\text{mge}(V)$ values across all seeds is at most 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Geometric Entropy AND Projective Geometry AND Communication Complexity Rank - Projective Varieties genus g MINIMAL GEOMETRIC ENTROPY Communication Complexity Rank - ccr(V) lower bound mge(V) Projective Geometry Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(matrix[j][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = 1 / matrix[i][i]
            for j in range(n):
                matrix[i][j] *= factor
            for k in range(n):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(n):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix

    def determinant(matrix):
        n = len(matrix)
        det = 1
        for i in range(n):
            det *= matrix[i][i]
        return det

    def communication_complexity_rank(V):
        # Placeholder function to compute ccr(V)
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 5) # Random rank for demonstration purposes
```

```

def minimal_geometric_entropy(V):
    # Placeholder function to compute mge(V)
    # This is a dummy implementation and should be replaced with actual logic
    return random.random() * 10 # Random entropy for demonstration purposes

n = random.randint(5, 40)
V = [[random.random() for _ in range(n)] for _ in range(n)]
ccr_V = communication_complexity_rank(V)
mge_V = minimal_geometric_entropy(V)

return {
    "metric_name": "mge(V) / ccr(V)",
    "metric_value": mge_V / ccr_V,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": mge_V >= 0.8 * ccr_V,
    "counterexample": "" if mge_V >= 0.8 * ccr_V else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

514235194, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 6.924292447914508, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 0.8017412009432732, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 1.2688092923276495, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 2.411714163665418, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 2.9409173378860562, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 2.0831319822238266, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 0.9688287801385265, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 0.6293222527667539, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 6.206237334465201, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 0.3761786165766057, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': True, 'counterexample': ''}

```

TRIAL: {'metric_name': 'mge(V) / ccr(V)', 'metric_value': 3.019608600154855, 'instances_tested': 1, 'RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed=71

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for both 'communication_complexity_rank(V)' and 'minimal_geometric_entropy(V)', returning random values. This does not provide a meaningful empirical test of the conjecture, as it lacks actual implementations of the required mathematical concepts.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed, $mge(V) < ccr(V) * 0.8$, which violates the conjecture's lower bound condition. | next: Investigate the counterexample 'mapping_undefined' to understand why the mapping is undefined and whether it can be resolved or if further theoretical work is needed to address this issue.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13658
2	preregistration	ollama_remote	glm4:latest	0	0	9648
3	novelty	ollama_remote	glm4:latest	0	0	8601
4	novelty	ollama_remote	glm4:latest	0	0	9826
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17062
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12635
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10469
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9509
9	critic	ollama_remote	glm4:latest	0	0	21518
10	judge	ollama_remote	glm4:latest	0	0	9613

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122539 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5a102f9f0fa2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5a102f9f0fa2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5a102f9f0fa2.tar.gz (if generated)